

5-2 · Docker Compose로 Spark 마스터-워커 구축 · 실습 1H

spark-submit으로 클러스터에 잡 제출

local[*] 이 spark:///... 로 바뀌는 순간

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 마스터 컨테이너의 `spark-submit` 으로 `pi.py` 예제를 제출한다
- ② `first_job.py` 를 넣고 제출해 master 값이 `spark://` 임을 본다
- ③ `--total-executor-cores` 로 제한하고 UI 의 Cores 를 비교한다
- ④ 자원을 과하게 요구해 `WAITING` 을 만들고, 두 잡을 동시에 돌린다

실행 중 마스터 UI 의 애플리케이션 이름을 누르면 5-1 의 Jobs·Stages 화면이 열립니다

다섯 단계 — 예제 · 내 스크립트 · 자원 · 실패 · 동시

모든 제출은 `docker compose exec spark-master /opt/spark/bin/spark-submit` 으로 — 마스터 컨테이너 안에서

1. pi.py	2. first_job.py	3. 자원 제한	4. 과한 요구(실패)	5. 동시 제출
예제 제출 Pi is roughly	cp 로 넣고 제출	cores 2 memory 1g	memory 8g WAITING	exec -d 로 두 개

spark-submit 에 붙이는 것들

마스터 주소	--master spark://spark-master:7077
예제	/opt/spark/examples/src/main/python/pi.py 10
파일 넣기	docker compose cp first_job.py spark-master:/tmp/
내 스크립트	/tmp/first_job.py
자원 제한	--total-executor-cores 2 --executor-memory 1g
동시 제출	docker compose exec -d spark-master ...

앞에 **docker compose exec spark-master /opt/spark/bin/spark-submit** 을 붙인다 — 마스터 컨테이너 안의 spark-submit

제출하고 로그를 읽는다 — 세 가지 증거

1

pi.py — Pi is roughly 3.14...

실행 중 8080 은 Running, 끝나면 Completed Applications 에 남는다

2

first_job.py — count = 500000

range(1, 1_000_001) 을 2배 해 1000000 초과만 센 결과

3

master: spark://spark-master:7077

5-1 에서는 local[*] 이었던 자리 — 진짜 클러스터에서 돈다는 증거

실행 중 UI 에서 이름을 누르면

4040 화면

Jobs·Stages

자원 옵션을 바꿔 Cores 열을 비교

제한 없음 — 기본값

워커의 자원을 최대한 쓴다

- 옵션 없이 /tmp/first_job.py 제출
- UI 의 Cores 열에 많이 잡힌다
- 다른 잡은 기다려야 한다

VS

--total-executor-cores 2

--executor-memory 1g 와 함께 제한한다

- Completed Applications 의 Cores 가 적다
- 여러 잡을 동시에 돌릴 때 나눠 쓴다
- 5단계 동시 제출의 열쇠

워커보다 큰 메모리를 요구한다

이론의 「자원이 모자라면 대기 상태로 멈춘다」를 직접 봅니다 — 40초 뒤 timeout 으로 끝납니다

timeout 40	memory 8g 요구	grep 경고	마스터 UI	낮춰 재제출
40초 뒤 자동 종료	워커는 2g 배치 불가	not accepted any resources	WAITING 상태로 남음	--executor-memory 2g 이하

경고를 만나면 — 오류가 아니라 「기다리는 중」

1

① 마스터 UI 에서 워커가 살아 있는지

Workers 표가 비어 있으면 자원을 줄 곳이 없다

2

② 남은 코어·메모리가 요청보다 큰지

--executor-memory 가 워커 메모리(2g)보다 크면 배치되지 않는다

3

③ 다른 잡이 자원을 다 쓰고 있지는 않은지

먼저 제출된 잡이 전부 차지했으면 끝날 때까지 기다린다

로그만 보면

멈춘 것처럼
보인다

두 잡을 동시에 — 코어를 2개씩 나눈다

--total-executor-cores 2 씩

두 애플리케이션이 동시에 RUNNING



VS

옵션 없이 제출했다면

첫 잡이 전부 차지해 둘째는 기다린다



이 1시간이 끝날 때 확인할 것

- 1 pi.py 로그 끝에 Pi is roughly 3.14... 가 찍힌다
- 2 first_job.py 로그에 count = 500000 이 찍힌다
- 3 master 가 spark://spark-master:7077 로 나온다
- 4 Completed Applications 에서 제한 건 쪽 Cores 가 적다
- 5 8g 요구는 not accepted any resources 반복 · UI 에 WAITING
- 6 exec -d 두 개가 동시에 RUNNING 으로 보인다

5-2 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

클러스터 명령을 한 장으로 정리하고, 5-1 과 5-2 의 차이를 표로 비교한 뒤 5-3 을 위한 상태를 확인합니다

✓ 명령 요약 여덟 줄

✓ 5-1(한 대) vs 5-2(클러스터)

✓ 자주 겪는 오류 네 가지

✓ 5-3 준비 — 마스터 UI 200 · MinIO 컨테이너

가져갈 것

UI 의 Completed 목록과 Cores 값